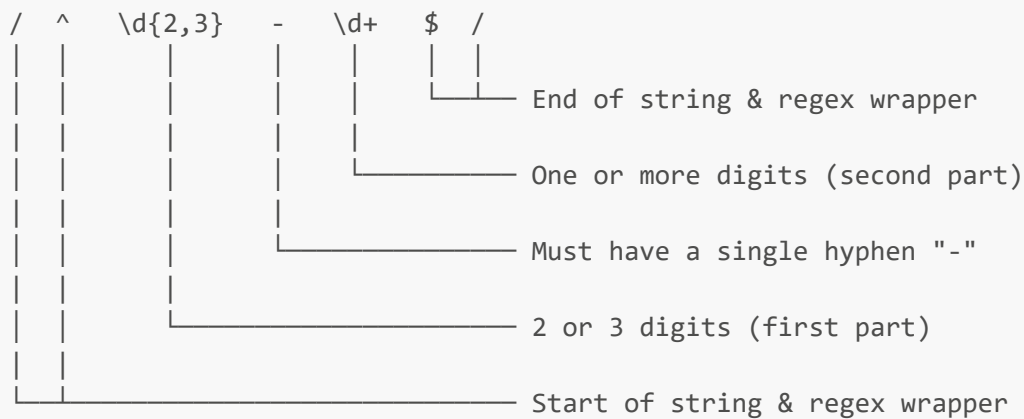


Validation and ESLint

Validation in regex for phonenumber

- eg. 09-1234556 and 040-22334455 are valid phone numbers
- eg. 1234556, 1-22334455 and 10-22-334455 are invalid



Regex Breakdown & Reference Guide

1. Breakdown of `/^\\d{2,3}-\\d+$/`

Symbol	Name	Meaning	Example Match
<code>/ ... /</code>	Delimiters	Encloses the regular expression in JavaScript	<code>/pattern/</code>
<code>^</code>	Start Anchor	Asserts the start of the string (no leading characters allowed)	Starts right at beginning
<code>\\d</code>	Digit Class	Matches any single digit from <code>0</code> to <code>9</code>	<code>0, 4, 9</code>
<code>{2,3}</code>	Range Quantifier	Matches the preceding token between 2 and 3 times	<code>09</code> (2 digits), <code>040</code> (3 digits)
<code>-</code>	Literal Character	Matches the exact hyphen character <code>-</code>	<code>-</code>
<code>\\d+</code>	Plus Quantifier	Matches 1 or more digits	<code>1234556, 22334455</code>
<code>\$</code>	End Anchor	Asserts the end of the string (no trailing characters allowed)	Ends right after digits

3. General Regex Cheat Sheet

Token	Meaning	Example
<code>\d</code>	Any digit <code>[0-9]</code>	<code>\d</code> matches <code>7</code>
<code>\D</code>	Any non-digit	<code>\D</code> matches <code>a</code> , <code>@</code>
<code>\w</code>	Any word character <code>[a-zA-Z0-9_]</code>	<code>\w</code> matches <code>A</code> , <code>3</code> , <code>_</code>
<code>\s</code>	Any whitespace (space, tab, newline)	<code>\s</code> matches
<code>.</code>	Any character (except newline)	<code>.</code> matches <code>x</code> , <code>9</code> , <code>%</code>
<code>*</code>	0 or more times	<code>a*</code> matches <code>"", "a", "aaa"</code>
<code>+</code>	1 or more times	<code>a+</code> matches <code>"a", "aaa"</code>
<code>?</code>	0 or 1 time (optional)	<code>https?</code> matches <code>http</code> , <code>https</code>
<code>{n}</code>	Exactly <code>n</code> times	<code>\d{4}</code> matches <code>2026</code>
<code>{n,m}</code>	Between <code>n</code> and <code>m</code> times	<code>\d{2,4}</code> matches <code>99</code> , <code>9999</code>
<code>[abc]</code>	Any character inside brackets	<code>[abc]</code> matches <code>a</code> , <code>b</code> , or <code>c</code>
<code>[^abc]</code>	Any character NOT inside brackets	<code>[^abc]</code> matches <code>x</code> , <code>1</code>
<code>(a b)</code>	Either <code>a</code> or <code>b</code>	<code>(cat dog)</code> matches <code>cat</code> or <code>dog</code>

4. Usage in Mongoose Schema

```
const personSchema = new mongoose.Schema({
  name: {
    type: String,
    minLength: 3,
    required: true
  },
  number: {
    type: String,
    minLength: 8,
    required: true,
    validate: {
      validator: function(v) {
        return /^\\d{2,3}-\\d+$/\\.test(v);
      },
      message: props => `\\${props.value} is not a valid phone number!`
    }
  }
});
```

⚙️ How `.test(v)` works:

validator: function(v) { return /^d{2,3}-d+\$/ .test(v) }

- v is the value the user typed in the phone number box.
 - .test(v) runs the regex.
 - If it matches the pattern → returns true (Mongoose saves it).
 - If it does NOT match → returns false (Mongoose rejects it with your custom message!).
-

1. What is ESLint? (The Big Picture)

- ESLint is the automated Style & Quality Inspector for JavaScript:
 - ESLint is the automated Style & Quality Inspector for JavaScript:
 - It reads your code without running it.
 - It catches hidden bugs (like unused variables or accidental global variables).
 - It enforces clean formatting rules (e.g. "In this company, we always use single quotes and 2 spaces")
-

The 4 Simple Steps to Set Up ESLint from Scratch:

- Step 1: install the Tools
 - **npm install eslint @eslint/js globals @stylistic/eslint-plugin-js --save-dev**

```
* **eslint**: The main inspection engine.  
  
* **@eslint/js**: Recommended default JavaScript rules.  
  
* **globals**: Tells ESLint that Node variables (like process.env and  
require) are allowed.  
  
* **@stylistic/eslint-plugin-js**: Rules for visual formatting (spaces,  
quotes, semicolons).  
  
* **--save-dev**: Tells npm this is only a development tool (not needed in  
live production).
```

- Step 2: The Rules File (eslint.config.mjs)
 - You create a single file named **eslint.config.mjs** in your project root. Think of this file like a Rulebook:

```

import globals from "globals";
import js from "@eslint/js";
import stylisticJs from "@stylistic/eslint-plugin-js";

export default [
  js.configs.recommended, // 1. Use standard recommended JS rules
  {
    files: ["**/*.js"],
    languageOptions: {
      sourceType: "commonjs", // 2. We use CommonJS (require/exports)
      globals: {
        ...globals.node, // 3. Allow Node globals (process, require,
etc.)
      },
      ecmaVersion: "latest",
    },
    plugins: {
      "@stylistic/js": stylisticJs,
    },
    rules: {
      "@stylistic/js/indent": ["error", 2], // Rule: Indent with 2
spaces
      "@stylistic/js/quotes": ["error", "single"], // Rule: Use single quotes
, ,
      "@stylistic/js/semi": ["error", "never"], // Rule: No semicolons at
end of lines
      "eqeqeq": "error", // Rule: Always use ===
instead of ==
      "no-trailing-spaces": "error", // Rule: No useless spaces
at end of lines
      "object-curly-spacing": ["error", "always"], // Rule: { name: 'Dan' }
instead of {name:'Dan'}
      "arrow-spacing": ["error", { "before": true, "after": true }],
      "no-console": "off", // Rule: console.log is
allowed in backend!
    },
  },
  {
    ignores: ["dist/**", "node_modules/**"], // Rule: Do not inspect
the frontend build or node_modules!
  },
];

```

- 📄 Step 3: Add the lint command in package.json

```

"scripts": {
  "start": "node index.js",

```

```
"dev": "node --watch index.js",  
"lint": "eslint ." // <=====  
}
```

-  Step 4: Run the Inspector!
 - npm run lint

Magic of eslint

1. ◦ ESLint tells you what to fix:
- When you run npm run lint, ESLint acts like a helpful guide.

```
index.js:15:3  error  Expected indentation of 2 spaces but found 4  
@stylistic/js/indent  
  
index.js:20:10 error  Strings must use singlequote  
@stylistic/js/quotes
```

- It tells you the exact file, exact line number, and what rule needs fixing.

-
2. ◦ Auto-Fixing (The Magic Trick! ✨):

run:-

```
npx eslint . --fix
```

- ESLint will automatically fix spaces, quotes, and semicolons across your entire project in 1 second!